ICS / OT CYBER RANGE

CPS Lab

Realistic, deployable industrial cyber-physical environment on Ludus
Architecture, connectivity and deployment reference

Version 2.0 · 13 November 2026
Supersedes: spec_lab_cps_v4.pdf
Reflects the current repository (7 VMs, NGFW-central Suricata, FastAPI console, OIDA kill chain)

1. Overview

CPS Lab is a 7-VM ICS/OT cyber range deployed in a single pass with Ludus and Ansible. It simulates a three-layer Purdue-model industrial network with live protocol simulators (S7comm, Modbus TCP, OPC UA), a central next-generation firewall (NGFW) that also runs Suricata, and a web console that drives both the attack side (OIDA) and the defense side (Suricata rules).

It is built for collaborative workshops: participants walk a complete ICS kill chain (reconnaissance, lateral movement, exploitation) while writing and testing detection rules against the traffic it produces.

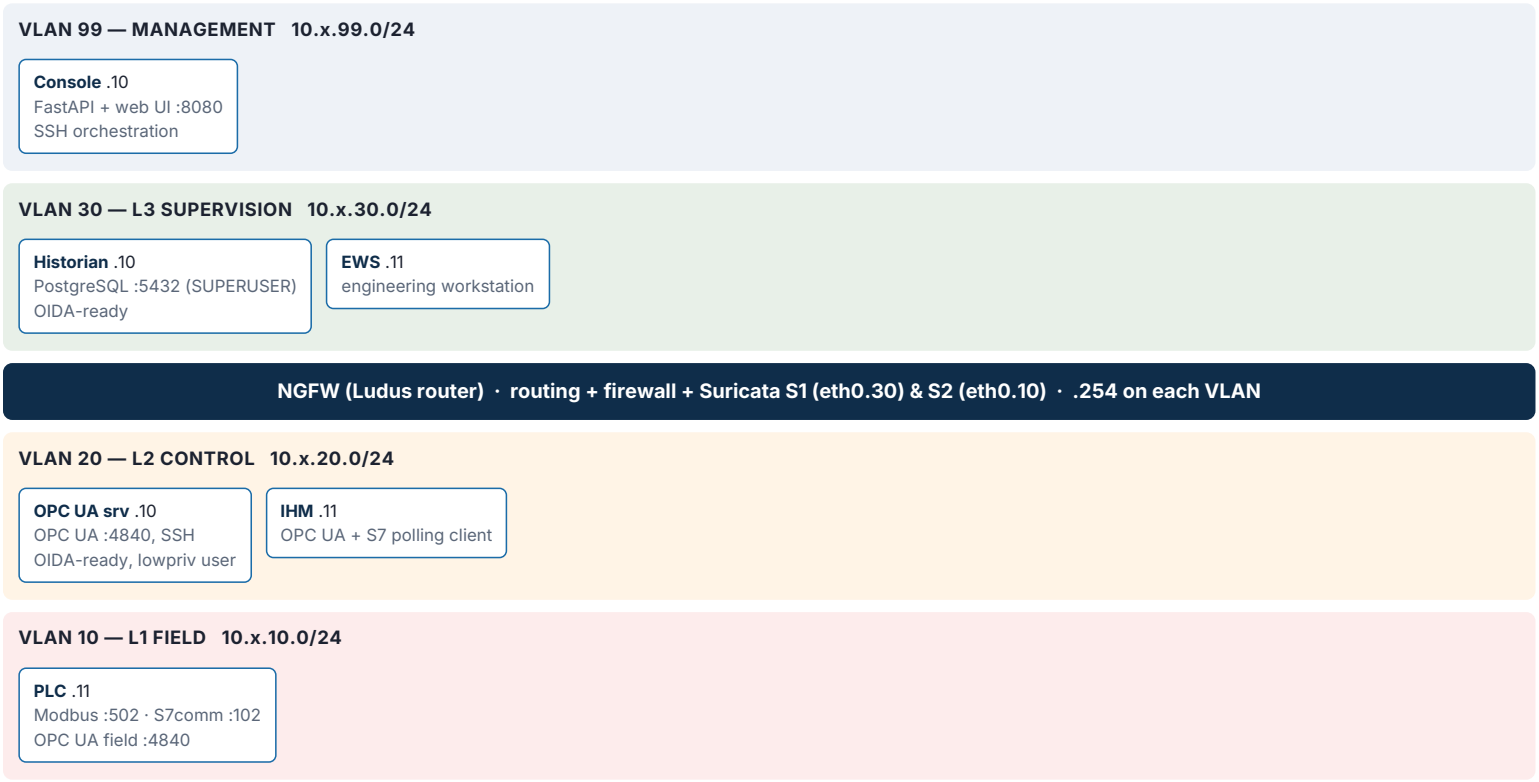
Key design decision

The Ludus range router **is** the NGFW. There are no separate probe VMs: Suricata runs directly on the router's VLAN sub-interfaces (`eth0.30` for L3/L2, `eth0.10` for L2/L1). The console interface keeps a full Purdue segmentation view for teaching clarity, even though the underlying infrastructure routes through one central node. This gap between the interface model and the infrastructure is intentional and documented, for consistency with how Ludus builds ranges.

> **Lab security notice.** This range is intentionally insecure (weak SSH password, SUPERUSER PostgreSQL, a committed disposable SSH keypair). It is meant for an isolated workshop range only. Never expose it to a production network and never reuse its credentials or keys.

2. Network architecture

Topology



The `10.x` second octet is assigned by Ludus per range (check it with `ludus range config get`). The NGFW is the gateway (.254) on every VLAN, so all inter-VLAN traffic transits it and is visible to Suricata.

VLANs and addressing

Layer	VLAN	Subnet	Hosts
Management	99	10.x.99.0/24	Console .10
L3 Supervision	30	10.x.30.0/24	Historian .10, EWS .11
L2 Control	20	10.x.20.0/24	OPC UA srv .10, IHM .11
L1 Field	10	10.x.10.0/24	PLC .11

Firewall matrix (default policy: REJECT)

Source	Destination	Ports	Action
L3 (30)	OPC UA srv (20)	tcp 22, 4840	ACCEPT
OPC UA srv (20)	PLC (10)	tcp 22, 102, 502, 4840	ACCEPT
IHM (20)	PLC (10)	tcp 102	ACCEPT
Console (99)	Historian (30), OPC UA srv (20)	tcp 22	ACCEPT
L3 (30)	L1 (10)	any	REJECT (segmentation gap)
any other inter-VLAN	—	any	REJECT

3. Suricata probes

Both probes run on the NGFW as independent systemd instances, sniffing a VLAN sub-interface with af-packet. Each has its own command socket for hot reloads and its own custom rules file, so what the console pushes to a probe is exactly what that probe loads.

Probe	Interface	Covers	Command socket	Custom rules
S1	eth0.30	L3 ↔ L2 (recon, SSH pivot)	/run/suricata-s1.socket	custom_s1.rules
S2	eth0.10	L2 ↔ L1 (exploitation)	/run/suricata-s2.socket	custom_s2.rules

Intra-VLAN traffic (for example IHM to OPC UA srv, both on VLAN 20) does not transit the NGFW and is therefore not visible to the probes. This is a deliberate consequence of the star topology.

4. Kill chain

Phase	From	To	Action	Visible on
1 · Initial compromise	(oral)	Historian	pg_execute_server_program on the SUPERUSER database	—
2 · Recon	Historian (L3)	OPC UA srv (L2)	oida opcua discover / browse	S1
3 · Lateral movement	Historian (L3)	OPC UA srv (L2)	SSH pivot as lowpriv	S1
4 · Recon	OPC UA srv (L2)	PLC (L1)	oida s7 identify / list-blocks	S2
5 · Exploitation	OPC UA srv (L2)	PLC (L1)	oida s7 stop / write, modbus write-coil	S2

The blue-team loop mirrors this: write a rule on the probe that watches the phase (S1 for phases 2 and 3, S2 for phases 4 and 5), run the attack from the console, and confirm the alert in the live `fast.log` stream.

5. Connectivity — the console backend

The console VM runs a FastAPI backend (port 8080) that bridges the web interface to the VMs over SSH using `asyncssh`. It serves the frontend and exposes the orchestration API.

API

Method	Endpoint	Purpose
POST	/api/oida/execute	Run an OIDA command on historian or opcua_srv
POST	/api/suricata/rules	Write custom_<probe>.rules on the NGFW and hot-reload
GET	/api/suricata/rules/{probe}	Read the current custom rules for a probe
GET	/api/status	SSH reachability of historian, opcua_srv, ngfw
WS	/ws/logs/{probe}	Live Suricata fast.log stream (tail -f over SSH)
GET	/health	Backend health

SSH key model

The lab uses a single pre-generated, disposable ed25519 keypair shipped with the roles. The console receives the private key; the targets authorize the public key (OIDA runner on Historian and OPC UA srv, management user on the NGFW). This deploys in one pass with no per-host fact passing and no dependence on VM provisioning order.

Connected vs demo mode

On load, the interface calls `/api/status`. If the backend answers, it runs in **Connected** mode (real SSH execution, live logs). If not, it falls back to **Demo** mode with canned OIDA output and simulated detection, for presentations without a live range. The mode badge is also togglable by hand.

6. ICS simulators

The ICS servers are not off-the-shelf daemons. Each is a small mock program shipped inside its role as a Jinja2 template (a `.py.j2` file). At deploy time the role renders it to a Python file on the VM, installs the library it needs in a local virtualenv, and runs it as a systemd service. The protocol logic lives in the script; the wire protocol comes from the library beneath it.

Service	VM	Role	Library	systemd unit
OPC UA server (Shopfloor)	OPC UA srv	cps-opcua-server	opcua (pip)	opcua-server
OPC UA field server	PLC	cps-opcua-field	opcua (pip)	opcua-field
S7comm server	PLC	cps-s7comm	python-snap7 (pip) + libsnap7 (apt)	s7comm-server
Modbus server	PLC	cps-modbus	pymodbus (pip)	modbus-server
IHM polling client	IHM	cps-ihm-client	opcua + python-snap7	ihm-client
OIDA (attacker tooling)	Historian, OPC UA srv	cps-oida-ready	oida-ics[all] (pip)	run on demand

OPC UA server

A complete server on the `opcua` library: a `Shopfloor` object with `Line_A` (Temperature, Pressure, Motor_RPM) and `Line_B` (Temperature, ValveState), writable variables, three security policies (None, Sign, SignAndEncrypt) with anonymous access allowed, and a background thread that drifts the values every 2 seconds. Listens on `0.0.0.0:4840`.

S7comm server and the snap7 client (two different roles for snap7)

The **server** side uses `snap7.server.Server()` to speak S7comm natively on port 102, registering data blocks DB1 (process data), DB2 (setpoints), DB10 (alarms) and DB100 (config) as raw byte arrays with realistic initial values. The **client** side appears twice: the IHM uses `snap7.client.Client()` to read DB1 every 10 seconds (the normal background traffic the blue team sees), and the attacker's client is OIDA, driven from the console against the same PLC. Both need the native `libsnap7` library in addition to the `python-snap7` pip wrapper.

7. VM summary and resource budget

#	VM	VLAN	Services	Kill-chain role	Comfortable
1	Historian	30	PostgreSQL, SSH	Compromise (1), OIDA runner (2)	2 vCPU / 4 GB
2	EWS	30	admin	Authorized L3 host	2 / 2 GB
3	OPC UA srv	20	OPC UA, SSH	Pivot (3), OIDA runner (4,5)	2 / 2 GB
4	IHM	20	OPC UA + S7 client	Background traffic	2 / 2 GB
5	PLC	10	Modbus, S7, OPC UA	Final target	2 / 2 GB
6	NGFW	router	routing, firewall, Suricata	Segmentation + detection	4 / 4 GB
7	Console	99	FastAPI + UI	Management plane	2 / 2 GB

Totals: about 16 vCPU / 18 GB RAM comfortable, 9 vCPU / 10 GB minimum, roughly 92 GB disk. Host requirement for Ludus itself: 32 GB RAM and 200 GB disk recommended.

8. Deployment summary

Full instructions are in `docs/setup.md` (install Ludus and build the template) and `docs/deployment.md` (deploy and test). In short, once Ludus is installed and the `debian-12-x64-server-template` is built:

```
git clone https://github.com/<your-org>/cps-lab.git
cd cps-lab

# register the ten roles with Ludus (once)
for role in ludus/roles/cps-*; do ludus ansible roles add -d "$role"; done

# load config, note the assigned second octet, deploy
ludus range config set -f ludus/range-config.yml
ludus range config get
ludus range deploy            # 15 to 25 min; follow with: ludus range logs -f

# verify, then open the console
curl http://10.<octet>.99.10:8080/api/status   # expect all three true
# browser: http://10.<octet>.99.10:8080
```

You do not install Suricata, snap7, OPC UA or OIDA by hand. The roles install everything inside each VM at deploy time.

Repository structure

```
cps-lab/
  README.md  LICENSE  .gitignore
  ludus/
    range-config.yml            7 VMs, VLANs, firewall rules
    roles/                      10 Ansible roles
    keys/                       disposable lab SSH key (committed on purpose)
  console/
    backend/   app.py, requirements.txt
    frontend/  index.html
  docs/
    setup.md        install Ludus + template
    deployment.md   deploy & test
    architecture.md full design document
```

9. Changes from the v4 spec

This version documents the lab as actually built, after a design review that fixed several inconsistencies between the v4 specification and a working deployment.

> **Per-probe rule files.** S1 and S2 now use `custom_s1.rules` / `custom_s2.rules` consistently across the Suricata config, the console backend and the reload script, so a pushed rule is actually loaded.

> **Per-instance command socket.** Each Suricata instance exposes its own `unix-command` socket, so rules hot-reload instead of forcing an engine restart, and the two instances no longer collide.

> **Working SSH key distribution.** Replaced a broken per-host fact mechanism with a single disposable keypair shipped in the roles, deployed in one pass regardless of VM order.

> **OIDA actually installed.** The oida-ready role now installs `oida-ics[all]` and the wrapper calls the real binary, so Connected mode runs real commands.

> **Robust SSH role.** The low-privilege SSH role ships its reload handler and no longer sets an `AllowUsers` list that could lock out the Ludus provisioning account.